STRIPE PROJECT

1. OLTP Data System

D2. OLTP Entity-Relation Diagrams

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D2 : OLTP ERD / v36 --- 2026/11/05.*

D2.1. Contexte

Ce document présente les diagrammes dits *ERD* (*Entity-Relation Diagram*) du modèle de données *OLTP*. Il s'agit de représenter les entités et les relations du modèle *logique*. Les éléments de construction du modèle *conceptuel* sont données en annexe A [D2-A]. La définition de référence du modèle *physique*, avec typages précis, contraintes et indexations, est donnée en annexe B [D2-B]. Les détails de conception (justification, etc.) sont donnés en annexe C [D2-C]. Une stratégie de réplication et de recouvrement est proposée en annexe D [D2-D] pour répondre à l'exigence BR1 [D0, §0.2.4].

Le modèle *logique* est scindé en différentes vues pour permettre une meilleure lisibilité :

- les vues globales regroupent les entités par fonctions et par dépendances,
- une vue du modèle centré sur les entités de référence partagées (référentiel géographique, monétaire, etc.).
- une vue du modèle centré sur les entités *Clients*,
- une vue du modèle centré sur les entités *Marchands*,
- une vue du modèle centré sur les entités *Transactions*,
- une vue du modèle centré sur les entités dédiées à la sécurité et à la conformité réglementaire,

Points particuliers du modèle

Le modèle transactionnel est conçu en priorité pour permettre une écriture rapide des transactions. La journalisation des lectures et des écritures est conçue pour ne pas ralentir le chemin critique du paiement (cf. [D1] - sujet d'intégration OLTP/NoSQL et OLTP/CDC). La même contrainte est appliquée sur le scoring de fraude (cf. [D1] - sujet d'intégration OLTP/Scoring).

Les autres fonctionnalités prises en compte n'interfèrent pas directement avec la procédure de paiement :

- gestion des alertes de sécurité,
- gestion des demandes de modification et des consentements,
- gestion des revue de fraude,
- gestion des abonnements,
- gestion des catalogue des marchands,
- gestion des plans de tarification des marchands,
- gestion des remboursements, litiges et rétro-facturations.

Limites du modèle

Le domaine fonctionnel du modèle de données est limité à la première phase du traitement des transactions financières (cf. [D0, §0.4]). La gestion des transactions avec les banques, des dépôts et des reversements aux marchands, ainsi que la plupart des produits et des services adjacents proposés par *Stripe*, sont hors du périmètre du modèle.

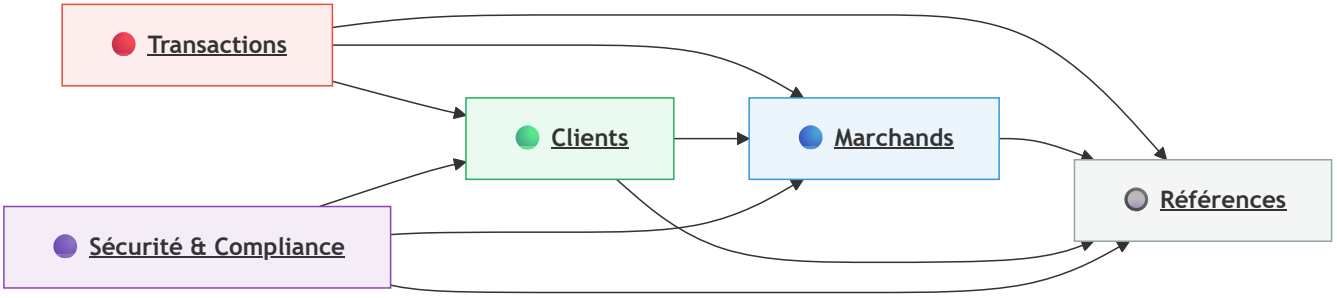
Le modèle ne partitionne aucune table et ne prévoit pas mise en cache : ces choix, ainsi que la préparation d'un éventuel sharding par marchand, sont justifiés en [D2-C, §1.C.4]. Les stratégies d'indexation sont définies dans le modèle physique [D2-B].

D2.2.1. Groupes fonctionnels et dépendances

Cette vue regroupe les entités du modèle par fonctions et par dépendances :

- *Références* :
  - tables autonomes d'énumération référencées par tous les autres groupes ;
- *Marchands* :
  - enregistrement des marchands et des tables dépendantes (produits, etc.),
  - ne dépend d'aucun autre groupe que celui des références ;
- *Clients* :
  - enregistrement des clients et des tables dépendantes (abonnement, etc.) ;
  - s'appuie sur le groupe des marchands ;
- *Transactions* :
  - enregistrement des transactions et des tables dépendantes (remboursement, fraudes, etc.) ;
  - s'appuie à la fois sur le groupe des marchands et celui des clients ;
- *Sécurité & Conformité* :
  - journaux et enregistrements diverses répondant aux exigences de sécurité (incidents, etc.) et de conformité réglementaire (consentement, demande de modification, etc.) ;
  - dépend des clients et des marchands (mais pas des transactions) ;

Noter que les vues suivantes reprennent les codes couleurs représentés ci-dessous.



> *Légende : Une flèche du groupe A vers le groupe B signifie que des tables de A référencent ou utilisent des tables de B.*

Noter que les vues des sections suivantes reprennent les codes couleurs représentés ci-dessus.

Points particulier du modèle

- *Pas de partitionnement :* les tables `Transaction`, `DataAccessLog`, `ChangeAuditLog` et `FraudScoringLog` restent des tables ordinaires, à clé primaire simple. Les tables filles ne portent donc aucune colonne de partition et leurs clés étrangères visent `transaction_id` seul.

D2.2.2. Groupes et tables

Cette vue reprend les groupes fonctionnels précédents en détaillant les tables qu'ils contiennent.
Les tables sont estampillées en fonction de leur nature, de leurs dépendances, et des contraintes d'écriture imposées (cf. Règles de gestion dans [D2-A]).

Les tables et leurs dépendances sont détaillées dans les vues suivantes.



> *Etiquettes de classe :*
>
> - ★ : Entité métier principale ;
> - ⊞ : Entité métier secondaire (extension directe ou indirecte d'une entité principale) ;
> - ☷ : Enumération ;
> - ▤ : Journal ;
> - 🔒 : Entité immuable (ne peut être ni modifiée ni supprimée après création) ;
> - ✕ : ne peut pas être journalisé (dans les journaux des accès et des écritures) ;

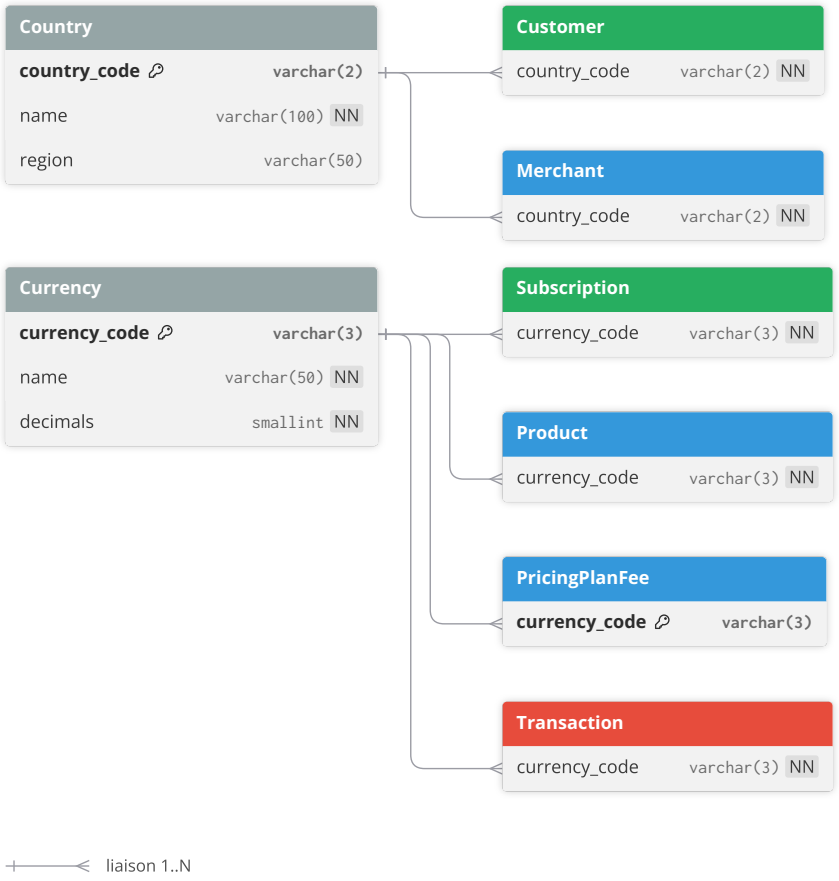
D2.3. Vues *Références*

Les tables de listes de référence partagées sont les suivantes (code couleur 🔘) forment deux sous-groupes :

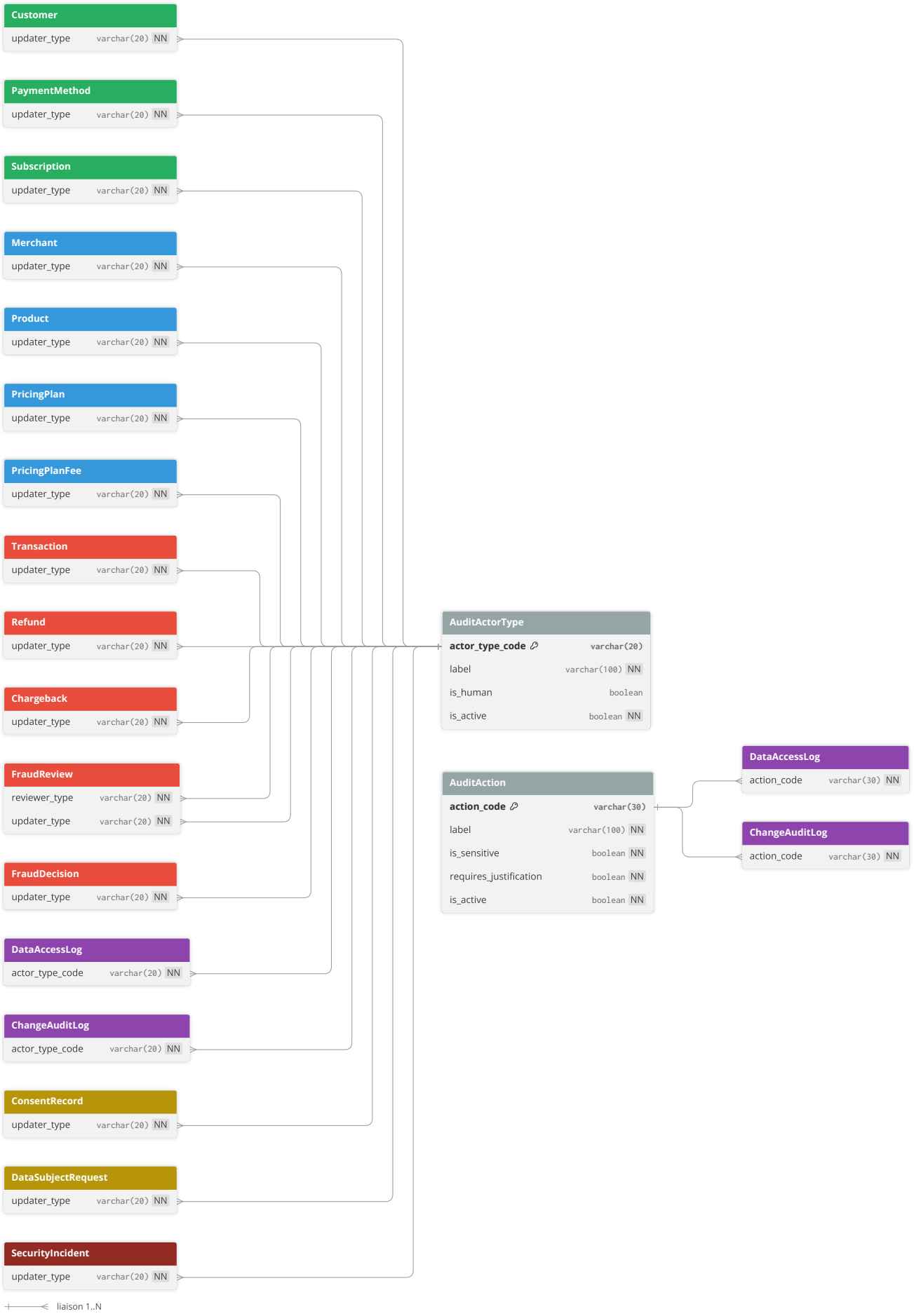
- listes de référence pour le coeur de métier : `Country`, `Currency` ;
- catégories d'auditables pour la sécurité et la conformité : `AuditActorType`, `AuditAction` ;
  - Types des acteurs auditables :
    - user,
    - system,
    - service,
    - merchant_api ;
  - Types des actions auditables :
    - Actions de lecture : read, export, login,
    - Actions d'écriture : create, update, delete, grant, revoke ;

Remarque : la plupart des listes de référence du modèle restent dans leur groupe d'utilisation tant qu'elles ne sont pas partagées par d'autres groupes (ex: *Regulation*, *Card Network*, etc.). Exceptionnellement, la table `AuditAction` suit l'autre table d'audit, `AuditActorType`, dans le groupe des références alors qu'elle n'est utilisée que par les tables de journalisation du groupe *Sécurité & Conformité*.

Coeur de métier

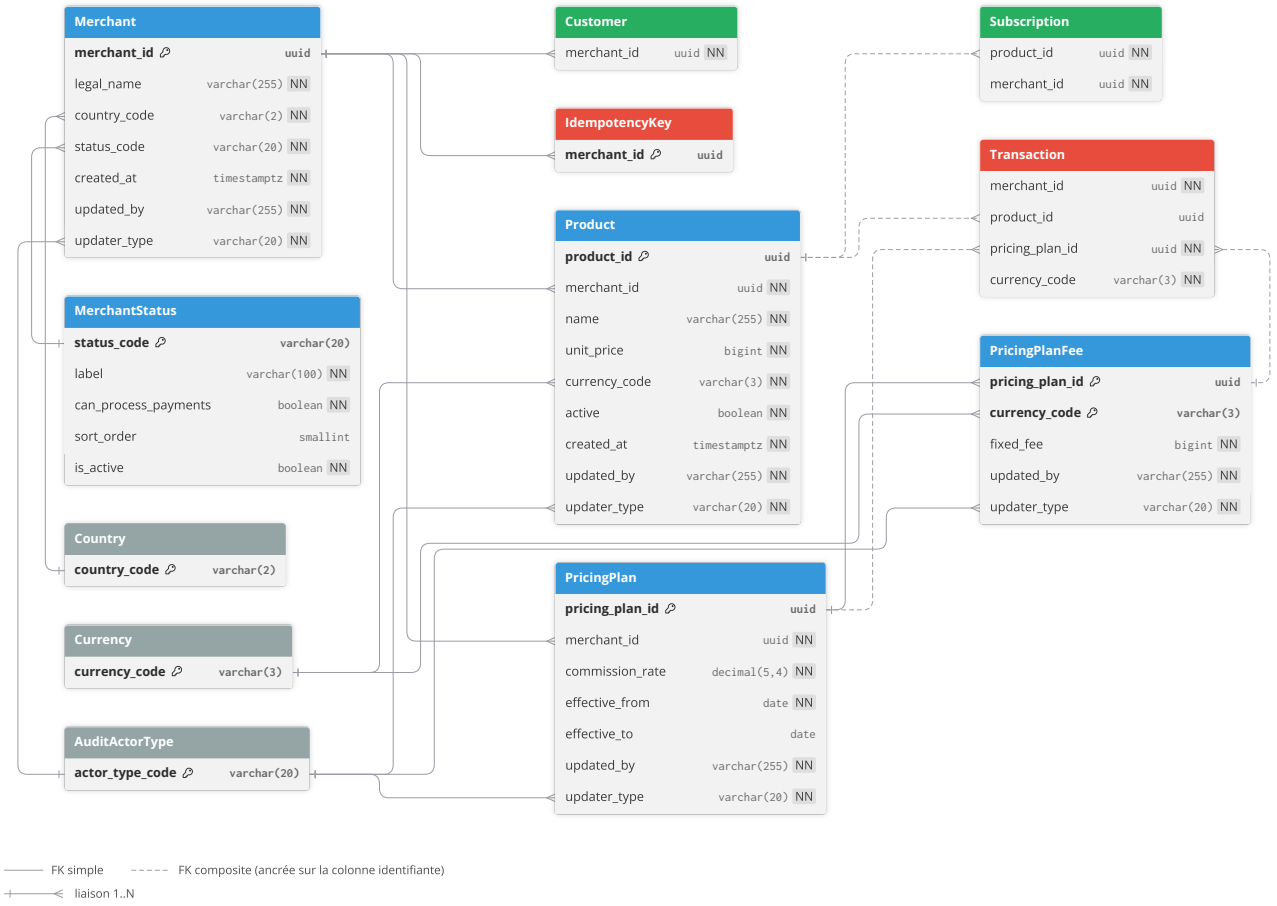


Catégories d'auditables



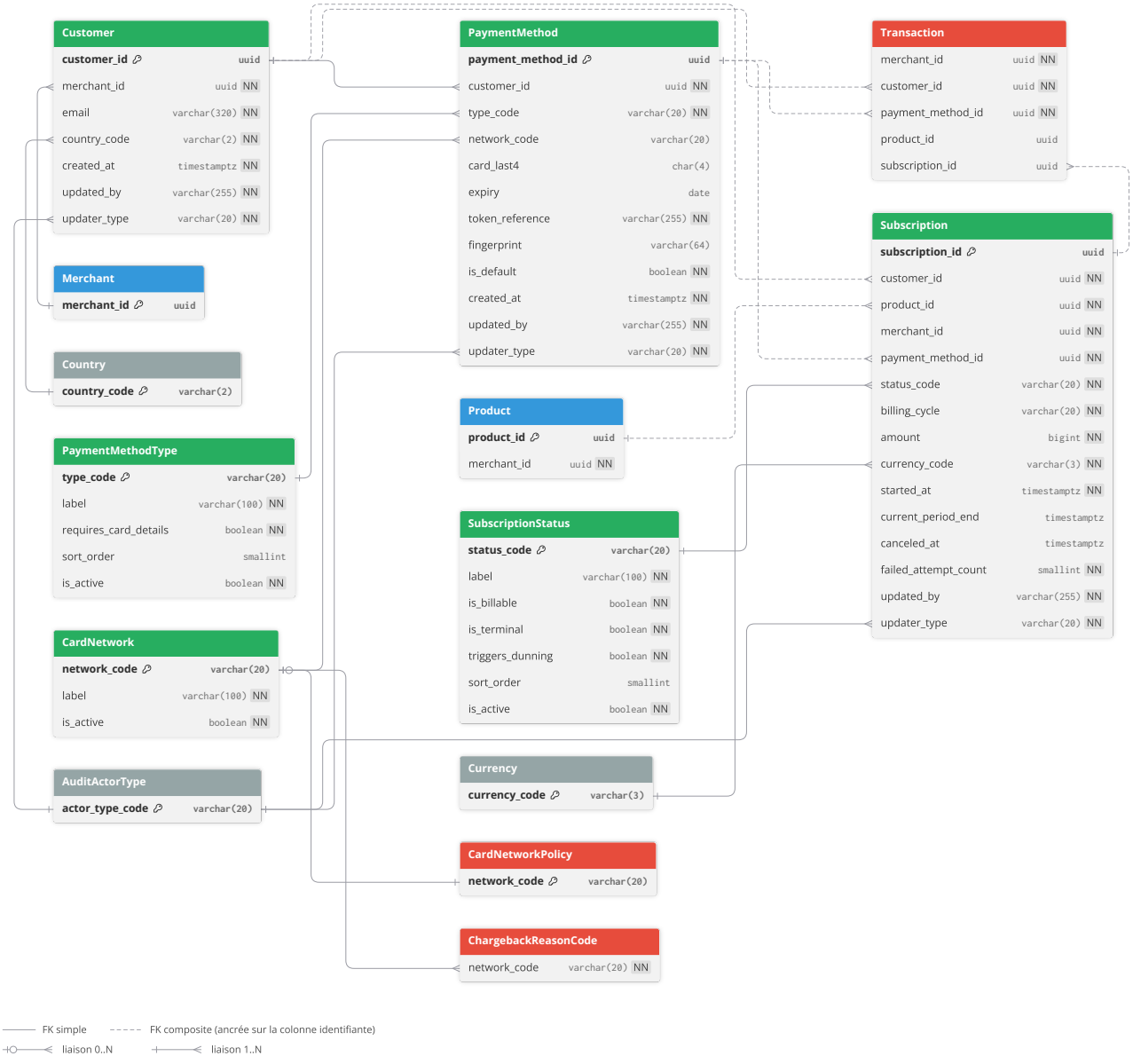
D2.4. Vue *Marchands*

Les tables de l'écosystème *marchand* sont les suivantes (code couleur 🔵) :



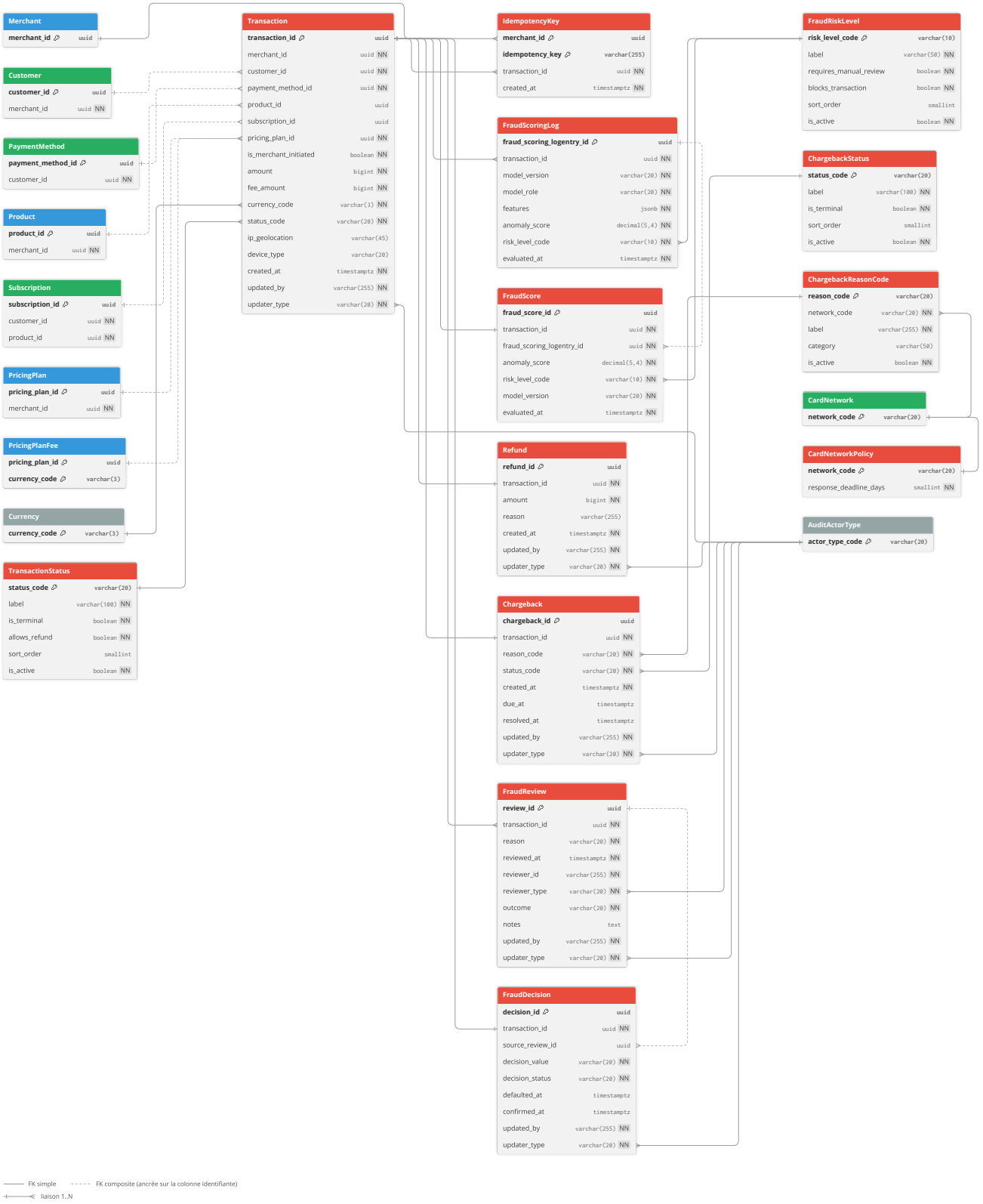
D2.5. Vue *Clients*

Les tables de l'écosystème *client* sont les suivantes (code couleur 🟢) :



D2.6. Vue *Transactions*

Les tables de l'écosystème des *transactions* sont les suivantes (code couleur 🔴) :



Remarque #1 : le délai de réponse du marchand (`response_deadline_days`) est spécifique au réseau de cartes et non au code de justification du litige (`ChargebackReasonCode`). Dans le cadre d'une normalisation stricte, il aurait fallu découpler le réseau de cartes lui même (ex: `CardNetworkPolicy`) du le code de justification, de la même manière que l'on a sorti `Regulation` de `DataSubjectRequest` .

Remarque #2 : la date limite de réponse du marchand (`due_at`) devrait être précalculée pour `Chargeback` , comme elle l'est pour `DataSubjectRequest` : `due_at = created_at + response_deadline_days` .

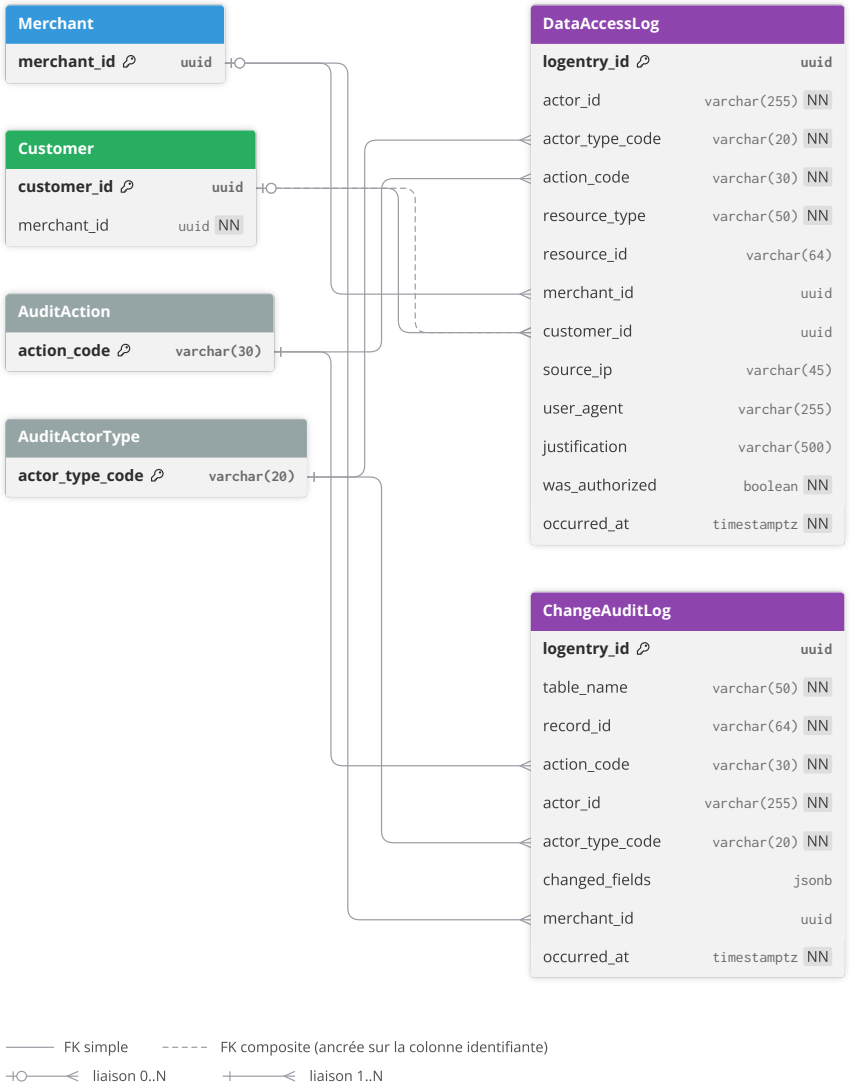
D2.7. Vue *Sécurité & Conformité*

Les tables dédiées aux questions de *sécurité* et de *conformité* sont les suivantes (codes couleurs 🟣, 🟡 et 🟠) :

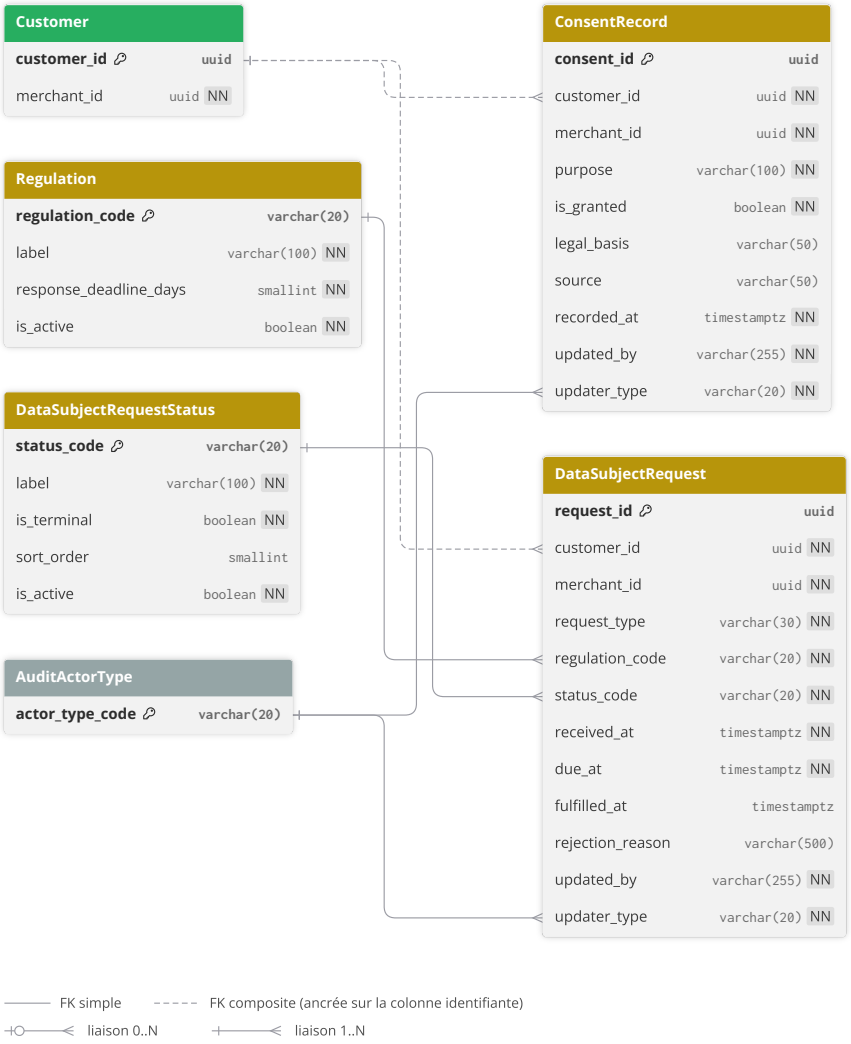
Les tables dédiées aux questions de *sécurité* et de *conformité* forment les sous-groupes suivants :

- Journalisation des accès et des écritures (🟣)
  - > concerne à la fois la sécurité et la conformité réglementaire ;
- Conformité et droits des utilisateurs (🟡)
  - > demandes de modification ou de suppression, consentements ;
- Incidents de sécurité (🟠)
  - > incidents de sécurité.

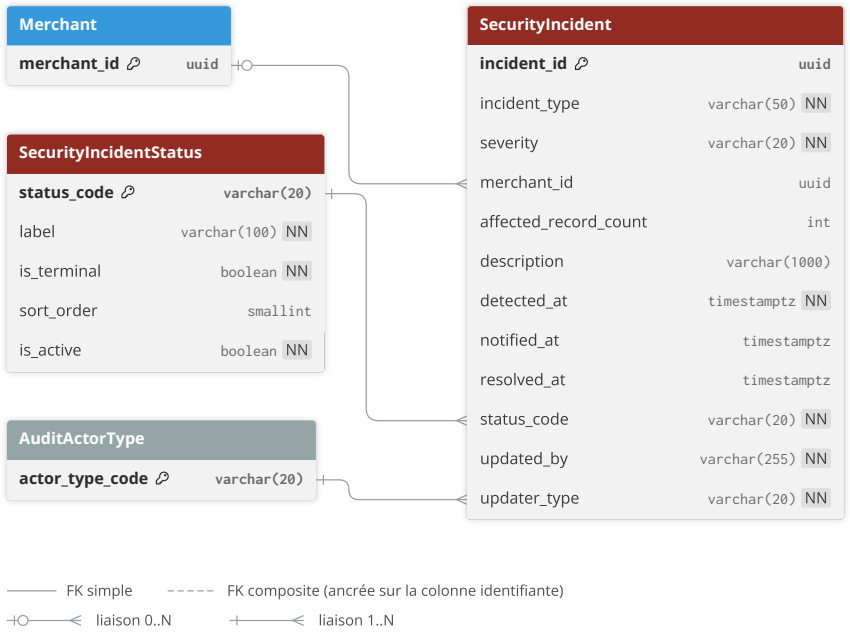
Journalisation des accès et écritures



Conformité et droits des utilisateurs



Incidents de sécurité



Couverture des exigences

Enjeux Stratégiques

#	Exigence	Couverture	Source
C1	Charges transactionnelles :	-	-
	- haute disponibilité	🟠	locale : réplication synchrone et bascule automatique ([D2-D] §1.D.2, §1.D.3) ; globale : système centralisé (N/D)
	- cohérence	🟢	natif SGBD (ACID) ; contraintes d'intégrité ([D2-B])
	- rapidité	🟠	indexation ([D2-B]), découplage de `Transaction` ([D2-C, §1.C.1.3]), journalisation hors chemin critique (§D2.1) ; manque partitionnement et cache
	- paiements (transactions)	🟢	`Transaction`, `IdempotencyKey`
	- remboursements (refunds)	🟢	`Refund`
	- rétro-facturations (chargebacks)	🟢	`Chargeback`
	- gestion des abonnements (subscription)	🟢	`Subscription`
	- systèmes évolutifs	🟠	montée en charge horizontale H/P ; préparation du *sharding* par marchand ([D2-C, §1.C.4])
	- systèmes capables de supporter de nouvelles fonctionnalités	🟢	3NF, tables de référence pour les énumérés, entités dépendantes découplées ([D2-C, §1.C.1])
	- systèmes capables de supporter la détection de fraude en temps réel.	🟠	`FraudScore`, `FraudScoringLog`, `FraudReview`, `FraudDecision` ; à compléter avec le scoring en temps réel dans [D1]
C2	Besoins analytiques	🔴	H/P [D2]
C3	Données N/S et S/S	🔴	H/P [D2]
C4	Intégration OLTP/OLAP/NoSQL	🔴	H/P [D2]
C5	Sécurité et conformité :	-	-
	- respect des réglementations	-	-
	--- RGPD	🟢	alerting, droits de modification, découplage `Regulation`
	--- CCPA	🟠	`Regulation` (délai de 45 j), `DataSubjectRequest` ; pas de fonction propre au CCPA
	--- PCI-DSS	🟢	PAN H/S (`last4`, `token_reference`, `fingerprint`)
	- mesures de sécurité robustes pour :	-	-
	--- protéger les données au repos/en transit	🔴	H/P [D2]
	--- gérer les contrôles d'accès	🟠	rôles et privilèges par fonction (application, écriture des journaux, lecture seule), journaux immuables ([D2-B]) ; *RBAC* applicatif H/P
	--- surveiller l'utilisation des données	🟢	`DataAccessLog`, `SecurityIncident`, `AuditAction`, `AuditActorType`

> *Légende : H/P : Hors Périmètre, H/S : Hors Système, N/A : Non Applicable, N/D : Non Distribué.*

Exigences d'affaire

#	Exigence	Couverture	Source
BR1	Intégrité transactionnelle	-	-
	- gros volumes de transactions	🟠	indexation de `Transaction` ([D2-B]) ; un seul primaire en écriture ([D2-D] §1.D.1) ; partitionnement et *sharding* manquants
	- latence minimale	🟢	`Transaction` auto-suffisante ([D2-C, §1.C.1.2]), index ciblés ([D2-B]), journalisation et scoring hors chemin critique (§D2.1) ; surcoût de la réplication synchrone intra-région ([D2-D] §1.D.2.2)
	- propriétés ACID	🟢	natif SGBD
	- cohérence des données en environnement distribué	🔴	H/P [D2]
	- intégrité des données en environnement distribué	🔴	H/P [D2]
	- transactions multi-documents	🟢	natif SGBD : transactions multi-tables et contraintes différées (ex : `IdempotencyKey` , `Refund`)
	- synchronisation des données en temps réel	🟢	réplication synchrone ([D2-D] §1.D.2) ; publication *CDC* pour les autres systèmes ([D2-B])
	- reprise après sinistre (*disaster recovery*)	🟠	réplique asynchrone inter-région, bascule manuelle, sauvegarde continue ([D2-D] §1.D.4, §1.D.5) ; outillage de la bascule inter-région non spécifié ([D2-D] §1.D.1)
BR2	Analytique évolutive et efficace	🔴	H/P [D2]
BR3	Gestion flexible des données	🔴	H/P [D2]
BR4	Intégration et cohérence des données	🔴	H/P [D2]
BR5	Sécurité et conformité	-	-
	- Mesures de sécurité robustes :	-	-
	--- Chiffrement	🔴	H/P [D2]
	--- Contrôles d'accès	🟠	cf. couverture C5
	--- Journalisation d'audit	🟢	cf. couverture C5
	- Respect des exigences réglementaires	🟢	cf. couverture C5
	- Reporting de conformité automatisé	🟠	délais matérialisés (`due_at`) ; production des rapports H/P
	- Surveillance en temps réel des failles de sécurité	🟢	cf. couverture C5

> *Légende : H/P : Hors Périmètre, H/S : Hors Système, N/A : Non Applicable, N/D : Non Distribué.*

Exigences Techniques

#	Exigence	Couverture	Source
TR1	Conception des modèles de données :	-	-
TR1/OLTP	- schémas normalisés	🟢	3NF ([D2], [D2-C, §1.C.1.2])
	- intégrité transactionnelle	🟢	clés, contraintes d'intégrité, contraintes différées, journaux immuables ([D2-B])
	- optimisation des performances	🟢	index ciblés et partiels ([D2-B]), découplage des entités dépendantes ([D2-C, §1.C.1.3])
TR1/OLAP	- schémas étoile/flocon pour requêtage et agrégation efficaces	🔴	H/P [D2]
	- tables pré-agrégées pour les requêtes courantes	🔴	H/P [D2]
TR1/NoSQL	- concevoir des schémas flexibles :	-	-
	--- supporter données non structurées	🔴	H/P [D2]
	--- permettre requêtage efficaces	🔴	H/P [D2]
	-- permettre récupération efficaces	🔴	H/P [D2]
TR2	Architecture du pipeline de données	🔴	H/P [D2]
TR3	Optimisation de l'évolutivité et des performances	-	-
	- gérer volumes de données croissants :	-	-
	--- monter en charge horizontalement	🔴	H/P ([D0, §0.4.3])
	- optimiser performances des systèmes :	🟠	locale : réplication synchrone, bascule automatique ([D2-D] §1.D.2, §1.D.3)
	--- stratégies d'indexation	🟢	index ciblés ([D2-B])
	--- stratégies de partitionnement	🔴	H/P ([D0, §0.4.3])
	--- de mise en cache	🔴	H/P ([D0, §0.4.3])
	- garantir une haute disponibilité :	-	-
	--- bases de données distribuées	🔴	H/P : SGBD centralisé ([D0, §0.4.3])
	--- sharding	🟠	H/P ([D0, §0.4.3]) ; préparé : UUID v7, cloisonnement par marchand, clé envisagée `hash(merchant_id)` ([D2-C, §1.C.4])
TR4	Cohérence et synchronisation	🟠	part OLTP : *CDC*, `ChangeAuditLog` ([D2-B]) ; pipeline, cohérence à terme et résolution de conflits H/P
TR5	Sécurité et conformité	-	-
	- Cadre de sécurité :	-	-
	--- Chiffrement	🔴	H/P [D2]
	--- Contrôles d'accès (RBAC)	🟠	cf. couverture C5
	--- Journalisation d'audit	🟢	`DataAccessLog` , `ChangeAuditLog` (écriture seule)
	- Respect des exigences réglementaires	-	-
	--- RGPD	🟢	cf. couverture C5
	--- PCI-DSS	🟢	cf. couverture C5
	--- CCPA	🟠	cf. couverture C5
	- Contrôles de conformité automatisés	🟠	délais matérialisés, contraintes ([D2-B]) ; exécution et alerting H/P
	- Fonctionnalités de reporting	🟠	données de conformité disponibles (`DataSubjectRequest` , `ConsentRecord` , `SecurityIncident`) ; reporting H/P
TR6	Analytique avancée et machine learning	🔴	H/P [D2] ; part OLTP : `FraudScoringLog`

Références

Annexes

- [D2-A] OLTP Data Conceptual Model
- [D2-B] OLTP Data Physical Model
- [D2-C] OLTP Design Notes
- [D2-D] OLTP Replication, Failover and Disaster Recovery Strategy

Livrables

- [D1] Data Architecture

Spécifications

- [D0] Stripe Project Introduction